

Algorithmic Trading System Development

Below are the detailed descriptions for the algorithmic trading system, including trading strategies, position sizing, money management rules, technical indicators, market watch, frontend, backend, AI Agents, etc. to develop the app by using the Interactive Brokers Paper Trading API. The app is to be developed in Python, leveraging the Polars library, DuckDB, and if required, any other libraries, databases, or frameworks, for efficient data manipulation and analysis. The app will execute paper / simulated trades across multiple asset classes, including stocks, indices, forex, and commodities from exchanges like NYSE, NASDAQ, LSE, and NSE.

Project Title:

- Algorithmic Trading System with Charting and Market Watch Dashboard

Objective:

1. Develop a modular and interactive web and mobile based Algorithmic Trading System using Python, integrated with the Interactive Brokers (IBKR) Paper Trading API and `ib_insync` for initial testing in a paper trading environment, with adaptability for live trading later.
2. The system includes a Market Watch Dashboard, charts, realtime /delayed /historical data fetching, real-time data monitoring, strategy execution, backtesting, trade management, and portfolio tracking.
3. The frontend will be built using:
 - **React.js** for a responsive, interactive, and modern web interface,
 - **Plotly.js** for interactive visualizations (e.g., candlestick charts, equity curves), and
 - **Tailwind CSS** for modern, clean styling , making the page look good and responsive across different devices.
 - **JavaScript** for dynamic behaviour and interactivity, allowing elements to change, data to be fetched without reloading the page, and complex user interactions.
4. The backend will use **FastAPI** for RESTful APIs and WebSocket support.
5. **Folder Path:**
C:\Users\Vincent_Pereira\Projects\Algo_Trading_Projects\Vincent Pereira\IBKR Paper Trading System
6. **IBKR Paper Trading API Details:**
 - **Host:** 127.0.0.1
 - **Port:** 7497
 - **Client ID:** 0
 - **Account No.:** DUK221396
 - **IBAPI Folder:** C:/TWS API/source/pythonclient
7. TWS API:

<https://www.interactivebrokers.eu/campus/ibkr-api-page/twsapi-doc/#api-introduction>

8. Development Environment:

- Use Visual Studio Code (VS Code).
- Target hardware: Lenovo Legion Pro 5 laptop.
- Ensure IBKR Trader Workstation (TWS) or IB Gateway is running in Paper Trading mode during development and testing.

Project Overview:

The Algorithmic Trading System will:

1. Connect to the IBKR Paper Trading API for live, delayed, and historical market data.
2. Implement three initial trading strategies, with a modular design for adding more strategies later:
 1. **Support & Resistance Strategy**
 2. **Trend-Following Strategy**
 3. **Hybrid Strategy (Mix of Support & Resistance and Trend-Following Strategies)**
3. Feature a Market Watch Dashboard for real-time monitoring of scrips, displaying prices, strategy signals, and technical indicators.
4. Market Watchlist based on:
 - (i) Highest Volatility Scrips
 - (ii) Highest Trading Volume of Scrips
5. Market Scanner
6. Create Technical Indicators by writing Python Functions to Calculate the following volume-weighted indicators using Polars:
 - 55-day Volume-Weighted Simple Moving Average (VW SMA) of Open:
 - Identifies the long-term trend direction.
 - Volume-weighted moving average of opening prices over 55 days.
 - 13-day VW SMA of Open:
 - Tracks medium-term price movements (optional for additional confirmation).
 - Volume-weighted moving average of opening prices over 13 days (optional).
 - 14-day Volume-Weighted Money Flow Index (MFI) of HLC Average:
 - Measures momentum based on the average of High, Low, and Close prices.
 - Volume-weighted Money Flow Index based on the average of High, Low, Close prices over 14 days.
 - 34-day VW SMA of MFI(14):
 - Smooths the MFI for trend confirmation.
 - Smoothing of the 14-day MFI over 34 days.
 - 21-day VW SMA of MFI(14):
 - Used as an exit trigger.

- Smoothing of the 14-day MFI over 21 days.
- 5-day VW SMA of High:
 - Defines short-term resistance levels.
 - Volume-weighted moving average of high prices over 5 days.
- 5-day VW SMA of Low:
 - Defines short-term support levels for exits.
 - Volume-weighted moving average of low prices over 5 days.
- 21-day Volume-Weighted Average True Range (ATR):
 - Measures volatility to set stop-loss and take-profit levels.
 - Volume-weighted Average True Range over 21 days.

7. AI Agents for Algo Trading

(i) Core Functional Agents

- Trade Execution Agent: Execute trades automatically based on market conditions and predefined strategies
- Risk Management Agent: Assess and mitigate risks by analysing market volatility, portfolio exposure, and other risk factors
- Market Analysis Agent: Analyse large datasets to identify trends, anomalies, opportunities in financial markets, identify market conditions (trending, sideways, volatile, etc.)
- Sentiment Analysis Agent: Process news articles, social media posts, and other textual data to gauge market sentiment and predict price movements
- IBKR API Agent

(ii) Specialised Agents

- Trend Following Agent
- Pair Trading Agent
- Quantitative Models Agent
- Support & Resistance Agent
- Gann Agent
- Fibonacci Agent
- Options Data Analyser Agent
- Other Strategy Agents

(iii) Supporting Agents

- Technical Indicators Agent
- Data Aggregation Agent: Collect and preprocess data from multiple sources, such as financial APIs, exchanges, and alternative datasets
- Compliance Monitoring Agent: Ensure all trades comply with regulatory requirements and internal policies

- **Backtesting Agent:** Simulate trading strategies on historical data to evaluate their effectiveness before deployment
 - **Portfolio Tracker Agent**
 - **Trade Position Tracking Agent**
 - **Multi-Agent Orchestration Frameworks:** Coordinate multiple AI agents to work together seamlessly, creating a cohesive trading ecosystem
8. Provide backtesting capabilities using historical data.
 9. Manage trades and portfolios with an "Auto" mode option for trade execution, including permission prompts for entry signals.
 10. Be optimized for local deployment, with potential for future cloud enhancements.

Core Components

1. **Data Integration**
2. **Strategy Execution**
3. **User Interface**
4. **Backtesting**
5. **Trade Management**
6. **Additional Features**

Technology Stack

- **Backend:**
 - **Python:** Core language.
 - **FastAPI:** For RESTful APIs and WebSocket support.
 - **ib_insync:** Python library for IBKR API interaction.
 - **Polars:** High-performance DataFrame operations.
 - **DuckDB:** Efficient market data storage and querying.
- **Frontend:**
 - **React.js** for a responsive, interactive, and modern web interface,
 - **Plotly.js** for interactive visualizations (e.g., candlestick charts, equity curves), and
 - **Tailwind CSS** for modern, clean styling, making the page look good and responsive across different devices.
 - **JavaScript** for dynamic behaviour and interactivity, allowing elements to change, data to be fetched without reloading the page, and complex user interactions.
- **Deployment:**
 - Local deployment on Lenovo Legion Pro 5, interfacing with IBKR TWS/IB Gateway.
- **System Configuration**

- Lenovo Legion Pro 5 Laptop:
 - ✓ AMD Ryzen 7 5800H
 - ✓ Radeon Graphics 3.20 GHz
 - ✓ 16 GB RAM
 - ✓ 1 TB GB SSD (500 GB Free SSD)
 - ✓ Nvidia GeForce RTX 3060 6 GB GPU
- **CUDA details:**
 - ✓ nvcc: NVIDIA (R) Cuda compiler driver
 - Copyright (c) 2005-2022 NVIDIA Corporation
 - ✓ Built on Wed_Sep_21_10:41:10_Pacific_Daylight_Time_2022
 - ✓ Cuda compilation tools
 - release 11.8, V11.8.89
 - Build cuda_11.8.r11.8/compiler.31833905_0

Key Features

1. Market Watch Dashboard

- Real-time display of scrips with prices, percentage changes, and strategy signals.
- Interactive visualizations using Plotly.js (e.g., candlestick charts, technical indicators).

2. Portfolio Overview

- Display account balance, open positions, recent trades, and metrics (e.g., profit/loss, Sharpe ratio).

3. Backtesting

- Simulate strategies on historical data, visualized with Plotly.js.

4. Strategy Signals

- Generate and display entry/exit signals from strategies.

5. Strategy Control

- Enable/disable strategies and add new ones modularly.

6. Trade Execution

- **Auto Mode:** When enabled, automatically send entry signals to IBKR Paper Trading Account. When disabled, prompt for permission for each entry signal (exit signals are always auto-executed).
- Place buy/sell orders via the IBKR API.

7. Risk Management

- Implement stop-loss, take-profit, and position sizing.

8. Logging and Reporting

- Log events, trades, and errors.

9. Real-Time Updates

- Use WebSockets for live data streaming from ib_insync to frontend.

Notes

- **Modularity:** New strategies can be added by subclassing BaseStrategy.
- **Auto Mode:** Entry signals prompt for permission when disabled; exit signals are always auto-executed.

I. Architecture & Design:

1. IBKR Interaction Model (Decision Point):

- **Suggestion:** Strongly consider centralizing *all* IBKR interactions (data subscriptions, order placement, account info fetching) into a dedicated module or microservice *within* your backend (ib_insync_manager or similar).
- **Reasoning:** This simplifies connection management (handling only one ib_insync instance), centralizes rate limit handling, improves error management related to IBKR, and makes the rest of the backend (API endpoints, strategy logic) agnostic to the specifics of the IBKR connection. It prevents potential conflicts if multiple parts of the backend tried to manage their own connections.
- **Modification:** Update the "Interaction Pattern" to reflect this centralized approach rather than allowing the general FastAPI backend direct access alongside a potential MCP server. Treat the IBKR interface as a core backend service.
- **Backend <-> IBKR:** Your FastAPI backend will maintain its *own* ib_insync connection for tasks like real-time data streaming, fetching account details, portfolio information, placing complex orders directly, or requesting historical data. This adds redundancy but might offer more control within the backend.
- **IBKR Interaction: ib_insync library used directly within FastAPI.**
 1. Manages connection to IB TWS / Gateway.
 2. Handles subscriptions for real-time market data (bars, ticks).
 3. Fetches historical data.
 4. Retrieves account information, positions, P&L.
 5. Places and manages orders (Market, Limit, Stop, Bracket etc.).
 6. Handles API events, errors, and disconnections.

2. Plugin System Deep Dive:

- **Plugin System:** For true modularity, consider a plugin-style architecture, where new strategies, indicators, or even data connectors can be added without modifying the core system code.
- **Suggestion:** Define the plugin interface more explicitly early on. How will strategies be discovered (e.g., specific folder, entry points)? What methods must a BaseStrategy subclass implement (e.g., on_bar, on_tick, calculate_signals, get_parameters)? How will parameters be passed and validated?

- **Reasoning:** A clear contract makes adding new strategies much smoother and less error-prone.
3. **Real-time Data Flow Resilience:**
- **WebSockets:** Excellent choice for pushing data to the frontend. Ensure your FastAPI implementation efficiently handles multiple client connections and broadcasts updates selectively (e.g., only sending updates for symbols a user is watching).
 - **Data Buffering/Throttling:** Be mindful of IBKR data rate limits. Implement logic to handle potential throttling or connection issues gracefully.
 - **Addition:** Implement robust reconnection logic for both the `ib_insync` connection to IBKR and the WebSocket connections to the frontend. Include exponential backoff strategies.
 - **Addition:** Implement heartbeat mechanisms on the WebSockets to detect stale/dead connections on both server and client sides.
 - **Consideration:** How will the system handle missed data during a disconnection/reconnection event (both from IBKR and to the frontend)? Will it attempt to fetch missing historical bars upon reconnect?
4. **Configuration Management:** You'll need separate configurations,
- IBKR connection details (account, host, port), symbols to subscribe to initially.
 - **FastAPI Backend:** Database connection (DuckDB), API settings, strategy parameters, risk settings, potentially its own IBKR credentials if using a separate connection.
 - **Frontend:** API endpoint URLs (for FastAPI), WebSocket URL, if required.
 - **Suggestion:** Use environment variables (.env files via libraries like `python-dotenv`) combined with Pydantic's `BaseSettings` for loading and validating configurations (IBKR details, API keys, database paths, strategy parameters, risk settings).
 - **Reasoning:** Keeps secrets out of code, simplifies configuration for different environments (dev, paper, live), and provides type safety.
5. **API Key/Credential Security:**
- WebSocket authentication. Add security (e.g., token authentication, origin validation) to prevent unauthorized connections or order submissions.
 - **FastAPI:** Secure your REST endpoints (JWT/OAuth2).
 - **Addition:** Ensure secure storage and handling of any necessary API keys (potentially for future data providers or notification services), not just IBKR credentials. Consider using environment variables or a secrets management system, especially if moving towards cloud deployment later.

II. Backend Enhancements:

1. Task Queuing (Celery/Redis):

- For computationally intensive tasks (complex backtests, report generation, potentially some asynchronous order checks) that shouldn't block the API or WebSocket communication, consider using a task queue like Celery with Redis or RabbitMQ as the broker. This integrates well with Python/FastAPI.
 - **Endorsement:** Strongly endorse considering Celery (or alternatives like RQ, Dramatiq) with Redis/RabbitMQ early on, especially for backtesting and potentially complex order monitoring or report generation tasks that shouldn't block the main API event loop.
 - **Reasoning:** Prevents API timeouts and keeps the system responsive.
2. **Database Considerations (DuckDB):**
- **Strengths:** Excellent for analytical queries on local data, integrates well with Polars/Pandas.
 - **Considerations:** Primarily designed for analytical workloads (OLAP). For very high-frequency updates or transactional needs, you *might* eventually explore other DB options (like TimescaleDB/PostgreSQL) *in addition* to DuckDB for specific use cases, but DuckDB is a strong start for local deployment and historical analysis.
 - **Data Storage Format:** Parquet files managed by DuckDB can be very efficient.
 - **Validation:** DuckDB + Polars is an excellent choice for efficient local data storage and analysis, especially for historical bar data and backtesting results using Parquet files.
 - **Consideration:** For extremely high-frequency *transactional* logging (e.g., every tick received, every internal state change), DuckDB might face write contention if not managed carefully. For *live trading* logging, consider if a more traditional transactional database (like PostgreSQL, potentially with TimescaleDB extension for time-series data) might be needed *in addition* to DuckDB for specific logging tasks, or ensure logging is buffered and written in batches. For the current scope (paper trading, local deployment), DuckDB is likely sufficient.
3. **API Security:**
- Implement robust authentication (e.g., JWT tokens via OAuth2 password flow) for your FastAPI endpoints, especially if the system becomes accessible beyond localhost.
4. **Asynchronous Operations:**
- Leverage async/await thoroughly throughout the backend (FastAPI, `ib_insync`, database interactions if the driver supports it) to maximize I/O concurrency and responsiveness.
5. **State Management:**
- **Addition:** Define how the backend maintains the *state* of strategies (e.g., active/inactive, current parameters, internal state variables like 'currently holding position'). This state needs to persist across restarts. Storing this in the database or configuration files is necessary.

III. Frontend Enhancements:

1. State Management (React):

- For a complex dashboard with real-time updates, React's built-in state management might become cumbersome. Consider libraries like Redux Toolkit, Zustand, or Jotai for more predictable and scalable state management across components.
- **Endorsement:** Strongly recommend choosing and implementing a dedicated state management library (Redux Toolkit, Zustand, Jotai) early. Real-time data updates across multiple components (dashboard, charts, portfolio) will become complex quickly.
- **Reasoning:** Improves maintainability, predictability, and performance compared to relying solely on React Context or prop drilling for global/shared state.

2. UI Component Library:

- While Tailwind provides utility classes, using a component library built on top of React/Tailwind (like Headless UI, Radix UI primitives + Tailwind, or even pre-built libraries like Flowbite, DaisyUI) can accelerate development and ensure UI consistency.
- **Endorsement:** Strongly recommend using a component library (Headless UI, Radix UI, Material UI, Ant Design, Chakra UI, etc.) alongside Tailwind CSS.
- **Reasoning:** Accelerates development, ensures accessibility, provides pre-built complex components (modals, dropdowns, tables), and improves UI consistency.

3. Charting Flexibility:

- Plotly.js is powerful. Ensure you leverage its capabilities for streaming updates for real-time charts rather than full redraws where possible.

4. Mobile Experience:

- React + Tailwind will give you a responsive web app. If a native mobile app experience is a future goal, keep React Native in mind as a potential path, sharing some logic with your React web app.

5. User Experience (UX):

- **Addition:** Consider adding user feedback mechanisms (e.g., loading spinners, toast notifications for success/error messages like 'Order placed', 'Connection lost', 'Strategy activated').
- **Addition:** Ensure clear visual distinction between paper trading and (potential future) live trading modes in the UI.

IV. Functionality & Features:

1. Backtesting Engine Enhancements:

- **Realism:** Enhance the backtester to simulate real-world conditions more accurately:
 - Slippage models (percentage, fixed, or based on volatility/volume).
 - Commission/fees structure (configurable per broker/exchange).
 - Latency simulation.
 - Order fill logic (how are market/limit orders simulated?).

- **Metrics:** Expand beyond basic metrics. Include Sharpe Ratio, Sortino Ratio, Max Drawdown, Calmar Ratio, Win Rate, Profit Factor, etc. `quantstats` is a great Python library for generating comprehensive performance reports.
- **Parameter Optimization:** Consider adding capabilities for strategy parameter optimization (e.g., grid search, random search, Bayesian optimization), possibly leveraging libraries like Optuna.
- **Endorsement:** The points on realism (slippage, commissions, latency) and expanded metrics (`quantstats` integration) are crucial for meaningful backtesting.
- **Addition:** Implement robust handling of historical data adjustments (splits, dividends) during backtesting. Ensure your data source provides adjusted data or implement the adjustment logic.
- **Addition:** Consider allowing backtests over specific date ranges and potentially comparing results of different parameter sets or strategies side-by-side.

2. Risk Management:

- Implement robust risk management rules *before* placing trades:
 - Position sizing based on account equity and risk per trade (e.g., 1% rule).
 - Max exposure limits per asset / overall.
 - Correlation checks (optional, avoid over-concentration in correlated assets).
 - Stop-loss and take-profit logic integrated into order placement (bracket orders if supported by IBKR API for the asset type).
- **Suggestion:** Implement *pre-trade checks* explicitly within the order placement logic. Before sending any order, verify:
 - Available buying power/margin.
 - Compliance with position sizing rules.
 - Maximum exposure limits.
 - Any kill-switch flags (global or strategy-specific).
- **Reasoning:** Adds a critical safety layer before interacting with the broker.

3. Data Management:

- **Historical Data Source:** Where will you get reliable historical data for backtesting? IBKR provides some, but you might need dedicated data vendors (e.g., Polygon.io, Alpaca) for longer history or specific asset classes. Build robust data ingestion and validation pipelines.
- **Data Cleaning:** Implement steps to handle missing data, outliers, or incorrect splits/dividends in historical data.

4. Alerting:

- Beyond dashboard signals, implement an alerting mechanism (e.g., email, push notifications via a service like Pushover, Telegram bot) for critical events: trade executions, errors, margin calls, strategy signal confirmations.

5. Logging and Monitoring:

- Expand logging: Log decisions, order placements, errors, API interactions, system performance. Use structured logging (e.g., JSON format) to make logs easier to parse and analyze.
 - Implement basic system monitoring (CPU, RAM, disk usage, network I/O) especially when running live.
6. **Order Management Robustness:**
- **Addition:** Implement logic to track order statuses beyond initial placement (Submitted, Pending, Filled, Cancelled). Handle partial fills.
 - **Addition:** Develop a mechanism to reconcile internal position state with the broker's reported positions periodically to catch discrepancies.
 - **Consideration:** How will bracket orders (entry + stop-loss + take-profit) be managed? `ib_insync` supports this, but requires careful state management.
7. **Strategy Development/Debugging:**
- **Addition:** Include features to visualize strategy behaviour on historical data charts during development (e.g., plotting indicator values, entry/exit points). Plotly is well-suited for this.
 - **Addition:** Allow "dry-running" strategies on live data *without* placing trades, just logging signals and potential P&L.

V. Non-Functional Requirements:

1. Testing Strategy:

- Implement unit tests (for individual functions/classes), integration tests (for interactions between components like API <-> DB), and potentially end-to-end tests (simulating user interaction) using frameworks like `pytest`.
- **Endorsement:** A strong testing suite (`pytest` is excellent) is non-negotiable for reliability.
- **Suggestion:** Prioritize:
 - Unit tests for core logic (indicator calculations, signal generation, risk rules).
 - Integration tests for API endpoints, database interactions, and potentially simulated `ib_insync` interactions (using mocking).
 - Consider basic end-to-end tests for critical paths like placing an order via the UI.

2. Deployment (Docker):

- **Docker:** Containerize your application components (Frontend, Backend API, MCP Server, Database) using Docker and Docker Compose. This drastically simplifies setup, ensures consistency across environments, and makes future cloud deployment easier.
- **Endorsement:** Using Docker/Docker Compose *from the start* is highly recommended, even for local deployment.
- **Reasoning:** Ensures environment consistency, simplifies dependency management, makes setup repeatable, and is the first step towards potential future cloud deployment.

3. Documentation:

- Document your code (docstrings), API endpoints (FastAPI auto-generates OpenAPI/Swagger docs), architecture, and setup process. This is invaluable for maintenance and future development.

4. **Logging:**

- **Suggestion:** Use a structured logging library (e.g., Python's built-in logging configured with JSON formatter, or libraries like loguru).
- **Reasoning:** Makes logs machine-readable for easier analysis, filtering, and potential integration with log analysis tools. Log key events: connections, disconnections, errors, order submissions, fills, signal generations, configuration changes.

VI. Hardware & CUDA:

1. **RAM:** 16GB is decent for starting but might become a limiting factor if you load very large datasets (e.g., years of tick data) into memory for backtesting or run many concurrent strategies. Keep an eye on memory usage.
2. **CUDA Utilization Plan:** You have an RTX 3060 and CUDA 11.8. How do you plan to leverage this?
 - **Machine Learning Strategies:** If you plan to implement strategies based on ML models (e.g., using PyTorch, TensorFlow), the GPU can significantly accelerate training and potentially inference.
 - **GPU-Accelerated Libraries:** Libraries like RAPIDS cuDF offer GPU-accelerated DataFrames, potentially speeding up data manipulation if your workflow involves massive datasets (though Polars is already highly optimized for CPU). Some backtesting or numerical libraries might have GPU support (e.g., VectorBT Pro, potentially custom NumPy code using CuPy).
 - **No Direct Use:** If your strategies are purely technical-indicator based and data manipulation is handled efficiently by Polars/DuckDB on the CPU, the GPU might not be directly utilized by the trading system itself, but it's available if your strategy complexity evolves.
 - **Clarification:** Be explicit about *if* and *how* you plan to use the GPU initially. If the first three strategies are purely based on standard technical indicators calculated via Polars/NumPy, the GPU won't be used.
 - **Suggestion:** If ML strategies *are* a near-term goal, allocate time to set up the CUDA environment correctly within your Python setup/Docker container and choose appropriate ML libraries (PyTorch, TensorFlow/Keras). If not, acknowledge the GPU is available for *future* expansion.
 - **Note:** Even complex indicator calculations using libraries like TA-Lib or Pandas TA are typically CPU-bound and highly optimized; Polars itself is very fast on CPU. GPU acceleration often requires specific libraries (cuDF, Numba, ML frameworks) and may only

show significant benefits with very large datasets or specific types of computation (linear algebra, neural nets).

General Considerations:

1. **Phased Development:** This is a complex project. Break it down into smaller, manageable milestones (e.g., Milestone 1: Connect to IBKR, stream data, display on basic UI. Milestone 2: Implement one strategy & backtesting. Milestone 3: Implement order placement & portfolio tracking, etc.).
2. **Error Handling:** Emphasize comprehensive error handling throughout the application – API errors, IBKR connection issues, data parsing errors, strategy execution errors, UI errors.
3. **Paper Trading vs. Live Trading:** Maintain a clear separation (config files, UI indicators, potentially code branches) between paper and live trading environments. Ensure thorough testing in paper trading before *ever* considering live deployment.